Cloud, IAM và mạng riêng

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Một ứng dụng chạy trên cloud gồm gì?

Hãy vẽ đường đi: người dùng -> DNS -> điểm vào public/load balancer -> ứng dụng -> database. Mỗi thành phần cần vị trí, quyền và đường mạng. Compute có thể là VM, container service hoặc serverless. VM cho quyền kiểm soát hệ điều hành nhiều hơn nhưng bạn phải quản lý vá lỗi và tiến trình. Dịch vụ managed giảm một phần công việc vận hành nhưng giới hạn tùy chỉnh và có mô hình giá riêng. Không có lựa chọn đúng cho mọi hệ thống.

Region là khu vực địa lý của cloud; zone là vùng tách biệt trong region theo thiết kế nhà cung cấp. Tính sẵn sàng cao đòi hỏi xem lỗi nào cần chịu được: tiến trình, máy, zone, region hay dependency bên ngoài. Hai bản sao ứng dụng cùng nằm trên một máy không giúp khi máy đó hỏng. Ngược lại, nhân bản qua nhiều zone làm tăng chi phí và độ phức tạp; nhu cầu kinh doanh quyết định mức đầu tư.

Storage khác nhau ở cách truy cập và tính chất: object storage phù hợp file/backup; block storage gắn với VM hoặc workload; database managed cung cấp truy vấn và chức năng dữ liệu. Đừng gọi mọi thứ là "ổ đĩa". Dữ liệu cần backup, restore và chính sách lưu giữ; replication không thay thế backup vì lỗi xóa nhầm có thể được nhân bản.

Ví dụ kiến trúc nhỏ

Ứng dụng ghi chú có một API và database. Giai đoạn đầu có thể chạy API trên VM và database managed hoặc cả hai ở local cho lab. Nếu cần public, đưa API qua HTTPS; database chỉ cho API kết nối và không mở công khai. Lưu cấu hình không bí mật riêng với secret. Ghi log request/error và giới hạn chi phí. Trước khi mở rộng, đo lưu lượng và yêu cầu phục hồi.

Bài tập giấy trước khi chạm cloud

Vẽ ba hộp compute, database và object storage. Ghi mũi tên kết nối, data nào nhạy cảm, component nào cần public và người dùng nào được phép thay đổi chúng. Giải thích vì sao database không cần public IP chỉ để API đọc/ghi. Bài tập này giúp tránh tạo tài nguyên tốn phí khi kiến trúc chưa rõ.

IAM, mạng và chi phí

IAM (Identity and Access Management) trả lời ai có thể làm gì với tài nguyên nào, trong điều kiện nào. Identity có thể là người dùng, service account hoặc workload identity. Permission là quyền hành động; role/policy nhóm quyền. Quyền tối thiểu nghĩa là cấp đủ cho công việc hiện tại, rồi mở rộng khi có bằng chứng cần thiết. Tài khoản dùng hằng ngày không nên có quyền quản trị toàn bộ chỉ để chạy một lab.

Mạng cloud thường gồm mạng ảo, subnet, route và bộ lọc truy cập. Subnet chia phạm vi địa chỉ; route quyết định đường đi; firewall/security group quyết định traffic nào được phép. Một dịch vụ không truy cập được có thể do DNS, route, firewall, listener, TLS hoặc lỗi ứng dụng. Hãy chẩn đoán theo lớp thay vì mở tất cả cổng để thử.

Chi phí cloud phụ thuộc thời gian chạy, loại tài nguyên, dung lượng và lưu lượng. Tài nguyên "nhỏ" vẫn có thể phát sinh phí nếu quên xóa. Trước lab, đọc trang giá hiện hành của dịch vụ bạn dùng, đặt ngân

sách/cảnh báo khi có, đặt tên tài nguyên dễ tìm và viết checklist cleanup. Không coi cảnh báo ngân sách là giới hạn cứng: nhiều nền tảng chỉ thông báo sau khi chi phí đã phát sinh.

Lab cloud có kiểm soát

Chỉ làm khi bạn đã chọn nhà cung cấp, tài khoản thử nghiệm và ngân sách. Bắt đầu bằng xem danh sách region, IAM identity hiện tại, service đang hoạt động và công cụ theo dõi chi phí. Tạo một tài nguyên nhỏ nhất cần cho bài học, ghi ID, region, quyền, giá dự kiến; xác minh rồi xóa sau buổi. Không thực hiện lệnh tạo/xóa từ tài liệu chung mà chưa kiểm tra đúng tài khoản và tên tài nguyên. Nếu chưa có tài khoản cloud, dùng sơ đồ và lab local để học khái niệm, rồi thực hành sau.

Tập quyết định kiến trúc với bốn câu hỏi

Trước khi chọn dịch vụ cloud, viết yêu cầu về traffic, dữ liệu, độ sẵn sàng và ngân sách. Ví dụ API ghi chú phục vụ nhóm 20 người, dữ liệu không quá lớn, chấp nhận vài phút ngừng dịch vụ trong lab và ngân sách thấp. Từ đó, một VM hoặc dịch vụ container managed cùng database managed nhỏ có thể là hai phương án đáng so sánh. Kubernetes nhiều node thường vượt nhu cầu và làm lab khó dọn hơn. Khi quy mô, yêu cầu uptime hoặc số nhóm tăng, xem lại quyết định.

Đối với mỗi phương án, trả lời: ai vá hệ điều hành và runtime; data ở đâu và backup thế nào; ai được phép deploy; khi một zone lỗi thì điều gì xảy ra; nếu quên tắt tài nguyên thì chi phí ra sao. Đây là cách dùng framework kiến trúc: chuyển tiêu chí chung thành câu hỏi cụ thể cho workload. Không có thiết kế "best practice" nếu chưa nói rõ bối cảnh.

Bài tập IAM và mạng có đáp án gợi ý

Ứng dụng cần đọc/ghi một database, CI cần triển khai image, còn người học cần xem log. Tách thành ba identity. App identity chỉ có quyền cần để kết nối database; CI identity có quyền deploy vào môi trường thử và đọc artifact cần thiết; learner identity đọc log, không cần quyền xóa database. Nếu tất cả cùng dùng một admin credential, khi một chỗ lộ thì phạm vi thiệt hại rất lớn.

Giả sử API có public endpoint nhưng database chỉ cho kết nối từ subnet/private network của ứng dụng. Khi API trả 500 vì không kết nối được DB, đừng mở database ra internet. Hãy kiểm tra địa chỉ cấu hình, DNS/private name, route, firewall ở cả hai đầu, credential và log database. Khi bạn xác định được lớp nào chặn kết nối, thay đổi đúng lớp đó rồi kiểm tra lại. Đường mạng thông không chứng minh credential hợp lệ.

Chi phí và cleanup là một phần của lab

Trước khi tạo tài nguyên, viết bảng: tên tài nguyên, region, dịch vụ, đơn vị tính phí, thời gian dự kiến, mức cảnh báo và lệnh/quy trình xóa. Sau buổi học, đối chiếu danh sách tài nguyên thực với bảng. Một VM đã tắt có thể vẫn còn storage; một load balancer, public IP hoặc managed database có thể tiếp tục tính phí tùy nhà cung cấp. Vì giá thay đổi, luôn dùng trang giá hiện hành của dịch vụ và vùng bạn chọn. Cảnh báo ngân sách giúp phát hiện, nhưng không nhất thiết tự chặn tiêu dùng.

Nếu Terraform quản lý tài nguyên, **destroy** cần được xem như một thay đổi có tác động: nhìn plan, kiểm tra tài khoản/region/workspace, xác nhận tên tài nguyên và dữ liệu. Nếu tài nguyên được tạo thủ công ngoài Terraform, destroy không biết để xóa. Sau thao tác, kiểm tra console/API và trang billing;

đừng chỉ tin vào dòng "complete" của một công cụ.

Đọc một thiết kế network cloud

Giả sử mạng ảo dùng dải **10.20.0.0/16**, trong đó subnet public cho load balancer và subnet private cho ứng dụng/database. Ký hiệu **/16** cho biết số bit đầu cố định của địa chỉ; subnet như **10.20.1.0/24** và **10.20.2.0/24** là các phần nhỏ hơn. Bạn chưa cần tính nhẩm mọi địa chỉ, nhưng phải hiểu rằng hai subnet có phạm vi địa chỉ và route khác nhau. "Public" không chỉ do tên subnet: cần xét route tới internet, public IP, bộ lọc mạng và dịch vụ có đang lắng nghe hay không.

Khi người dùng truy cập **https://notes.example**, DNS trỏ tới điểm vào. TLS có thể kết thúc ở load balancer hoặc proxy theo thiết kế. Điểm vào chuyển tới API trong subnet private; API nói chuyện database qua đường private. Nếu API lỗi kết nối DB, theo lớp: host/port cấu hình; DNS nội bộ; route và firewall; DB đang chạy; xác thực; giới hạn kết nối. Không mở port DB cho **0.0.0.0/0** chỉ để thử; nó làm tăng phạm vi tấn công mà không chứng minh nguyên nhân.

Bài tập CIDR và luồng traffic

Vẽ hai subnet **10.20.1.0/24** và **10.20.2.0/24**, đặt API ở subnet thứ nhất, DB ở subnet thứ hai. Ghi rule ý tưởng: chỉ API được kết nối tới cổng DB; người dùng chỉ tới cổng HTTPS của điểm vào. Sau đó nêu ít nhất hai lý do kết nối vẫn có thể lỗi dù rule này đúng: DB không lắng nghe, credential sai, DNS sai hoặc route thiếu. Bài tập muốn bạn tách **network reachability** khỏi **authentication**.

Thiết kế cloud từ luồng dữ liệu và quyền

Trước khi chọn AWS/Azure/GCP service, vẽ request: Internet -> edge -> ứng dụng -> DB/queue/storage, cộng thêm đường quản trị và monitoring. Đánh dấu dữ liệu nào rời vùng riêng, danh tính nào gọi API nào và failure domain của từng dịch vụ. VM, container, App Service hoặc serverless đều có compute; managed DB chuyển một phần trách nhiệm patch/HA cho provider nhưng bạn vẫn sở hữu schema, quyền, backup policy và kiểm tra restore. “Managed” không có nghĩa không cần vận hành.

Shared responsibility thay đổi theo dịch vụ. Trên VM, bạn thường chịu OS hardening và patch; trên managed app platform, provider làm nhiều hơn ở host nhưng app code, dependency, identity và dữ liệu vẫn thuộc bạn. Cần đọc thỏa thuận cụ thể của provider/service, không dùng một bảng chung để kết luận mọi cấu hình an toàn. Ở hybrid, đường on-prem -> cloud có thêm VPN/ExpressRoute/Direct Connect, DNS, firewall, route và identity trust; lỗi ở bất kỳ lớp nào có thể giống lỗi ứng dụng.

IAM: chủ thể, hành động, tài nguyên và điều kiện

Mỗi quyết định quyền có bốn phần: **ai** (human/service identity), **làm gì** (action), **trên tài nguyên nào**, **trong điều kiện nào**. Ví dụ CI chỉ cần đẩy image vào một registry repo, không cần quyền sửa VNet. App chỉ cần đọc một secret hoặc ghi một queue, không cần owner cả subscription/account. Quyền rộng làm giảm thời gian gỡ lỗi trước mắt nhưng mở rộng blast radius. Dùng managed/workload identity và token ngắn hạn khi nền tảng hỗ trợ; tránh cất access key lâu dài trong repo hoặc máy cá nhân.

RBAC cho phép có điều kiện không đồng nghĩa network đã mở; network mở không đồng nghĩa identity có quyền. Nếu Azure SQL trả lỗi, phân biệt DNS/private route/TCP/TLS/auth/database permission. Nếu

cloud API trả **Forbidden**, xem principal thực của workload (khác principal bạn dùng ở portal), role scope, deny policy và thời gian propagation. Ghi lại identity qua log/audit, không đoán dựa vào tên Pod.

Mạng riêng qua một kịch bản

Notes API chạy trong subnet ứng dụng; DB dùng private endpoint. App gọi hostname DB chuẩn. DNS phải trả IP private từ nơi app chạy; route và NSG/firewall phải cho app tới IP/cổng; DB auth phải chấp nhận identity; DB engine phải phục vụ query. Nếu bạn hardcode IP private vào app, thay endpoint có thể phá kết nối và TLS hostname validation. Nếu cùng hostname trả IP public trên laptop nhưng private trong VNet, đó có thể là split DNS dự kiến; dùng **nslookup**/resolver từ đúng runtime để xác minh.

Thiết kế CIDR không chồng lấp với on-prem và các VNet/VPC kết nối. Public subnet/private subnet là mô tả đường route và exposure, không phải nhãn bảo mật tự động. NAT Gateway giúp egress có kiểm soát nhưng không thay firewall/identity. Load balancer có health check nhưng health check **GET /** nông có thể đưa traffic tới app không ghi DB được. Cloud firewall rule và Kubernetes NetworkPolicy ở hai lớp khác nhau; khi điều tra phải biết packet đi qua lớp nào.

Lab thiết kế không tốn tiền

Đề bài: vẽ một môi trường staging cho Notes API gồm edge HTTPS, 2 replica app, DB managed, object storage backup và telemetry. Giả định cần riêng tư dữ liệu, một zone có thể hỏng, ngân sách hạn chế. Tạo bảng cho từng thành phần: dịch vụ, vùng mạng, identity, inbound/outbound, secret, dữ liệu lưu, backup, điểm hỏng và ước lượng chi phí cần kiểm tra trong calculator chính thức. Không tạo cloud resource ở bài này.

Bước 1: vẽ hai luồng: request người dùng và đường backup/restore. **Bước 2:** tạo quyền tối thiểu: app đọc một secret, truy cập DB và ghi object backup chỉ nếu thật sự là app phụ trách; CI không được đọc dữ liệu DB. **Bước 3:** đặt failure domain: hai app replica trên hai zone có ích nếu DB/edge không cùng điểm hỏng, nhưng cũng làm tăng chi phí. **Bước 4:** lập kế hoạch kiểm chứng: DNS private từ app, HTTP smoke, DB restore sang môi trường cách ly, mất một zone, alert. **Bước 5:** ghi tradeoff: khi ngân sách chưa cho multi-zone DB, RTO/RPO nào chấp nhận và rủi ro nào còn lại.

Lời giải định hướng: đừng bắt đầu bằng "dùng AKS/EKS". Nếu chỉ một API nhỏ, managed app platform + DB managed có thể giảm gánh vận hành. Nếu yêu cầu nhiều microservices, policy và đội có năng lực, K8s có thể hợp lý. Chọn sau khi định lượng tải, độ tin cậy, kỹ năng và chi phí. Tính cả log ingestion, network egress, storage snapshot và tài nguyên idle; chi phí cloud không chỉ là VM giờ chạy.

Câu hỏi phỏng vấn có lời giải

Private Endpoint đã tạo nhưng app vẫn tới public IP. Kiểm tra private DNS zone/link/resolver trong runtime app và cache; endpoint object riêng chưa ép mọi DNS trả IP private. **App có role nhưng vẫn Forbidden.** Xác định principal thực, action/resource scope và deny/propagation; đừng thêm Owner ngay. **Hai VM ở hai zone nhưng DB một zone.** Ứng dụng chưa chịu được mất zone DB; xét DB HA, restore và dependency khác. **Lab xong không dọn.** Tài nguyên có thể tiếp tục tính phí; phải có tag, budget alert, IaC destroy plan và kiểm tra sau dọn.

Nguồn: Azure Well-Architected identity, Azure mission-critical networking, AWS IAM introduction.

Quyền tối thiểu qua ma trận thao tác

Đừng viết "app cần quyền Storage" chung chung. Lập ma trận: **notes-api** cần đọc secret **db-password** và ghi blob vào bucket **notes-attachments**, không cần xóa bucket hay đọc secret của service khác; **ci-staging** cần push image vào repo staging và cập nhật manifest staging, không cần quyền production; người trực ca cần xem log/metric và một quyền break-glass có audit. Từ ma trận này tạo role/policy theo tài nguyên/scope, rồi kiểm tra bằng một thao tác được phép và một thao tác bị từ chối. Nếu chỉ kiểm tra đường happy path, một quyền quá rộng sẽ không bị phát hiện.

Identity có thể là user, service principal, managed/workload identity; credential có thể là password/key/token/chứng thực tạm. Secret manager bảo vệ nơi lưu và cấp phát, nhưng app vẫn có thể lộ secret qua log, exception hoặc quyền debug/exec. Lập kế hoạch rotate: secret cũ/mới cùng tồn tại trong một khoảng nếu cần, cập nhật consumer, xác nhận kết nối mới, thu hồi cũ, kiểm tra audit. Một rotation không có danh sách consumer có thể gây outage.

Bài toán hybrid CIDR có lời giải

On-prem dùng **10.10.0.0/16**; VNet cloud A đề xuất **10.10.20.0/24**. Dải cloud nằm trong on-prem, route qua VPN không thể phân biệt đích bằng prefix đơn giản; cần chọn CIDR không chồng lấp hoặc thiết kế NAT/renumber với tradeoff rõ. Nếu cloud B dùng **10.20.0.0/16**, không chồng lấp on-prem trong ví dụ; vẫn phải kiểm tra mọi site khác, peering/transit và route. Đừng chỉ "mở NSG 1433" rồi kỳ vọng DB nối được: DNS private, route đi/về, firewall và auth là các điều kiện độc lập.

Chi phí và an toàn lab

Một tài nguyên luôn chạy 24/7 trong tháng khoảng 730 giờ, nhưng hóa đơn còn storage, network egress, logging, snapshot và license. Trước khi tạo lab cloud, ghi region/SKU, số giờ dự kiến, tài nguyên phụ sẽ tự tạo và cách xóa. Đặt budget alert nhưng hiểu alert không tự ngắt tài nguyên trừ khi có automation riêng. Dùng IaC để tạo/dọn trong subscription/project lab riêng, tag mọi resource, kiểm tra sau destroy vì object ngoài state/manual có thể còn. Nếu muốn học Private Endpoint mà chưa có ngân sách, bài thiết kế DNS/route trên giấy có giá trị hơn tạo một cluster đắt rồi không hiểu chi phí.

Case phỏng vấn

Một app mới được cấp Owner subscription để "fix Forbidden"; sau đó gọi DB vẫn timeout. Vì sao? Owner là quyền control plane rộng, còn timeout xảy ra trước auth DB và có thể do DNS/route/firewall. Trả quyền về tối thiểu, điều tra từ runtime theo lớp. Một role rộng không giúp TCP tới nơi và tạo rủi ro lớn hơn. Nguồn: Azure RBAC overview, AWS IAM best practices.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.

- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html
- https://learn.microsoft.com/azure/role-based-access-control/overview
- https://docs.cloud.google.com/iam/docs/overview